AI with Rav

Learn AI the way you actually think.

DAY 1 of 30

OPEN SOURCE CONTRIBUTION

My First Merged PR in a Library You Already Use

WHAT YOU'LL LEARN TODAY

- How to find a bug nobody has claimed
- Why one word changed 29 lines
- How to write a PR a maintainer says yes to
- The exact steps, so you can do it this week

My First Merged PR in a Library You Already Use

> In One Line: I found a real bug in joblib — a library scikit-learn depends on — fixed it in one line, and it merged in nine days. Here is exactly how.

Why this video exists

Most "contribute to open source" advice stops at *"find a good first issue."* Then you open the issue tracker, see forty people already commenting **"can I work on this?"**, and close the tab.

This is the other version. One real bug, in a library with **4,380 stars** that ships inside scikit-learn, found by using the library rather than by trawling for tasks.

The library

joblib does two things almost every Python data project needs: it saves objects to disk and runs loops in parallel. If you have ever written **joblib.dump(model, "model.pkl")**, you have used it.

It is a dependency of scikit-learn, so it runs on far more machines than its star count suggests.

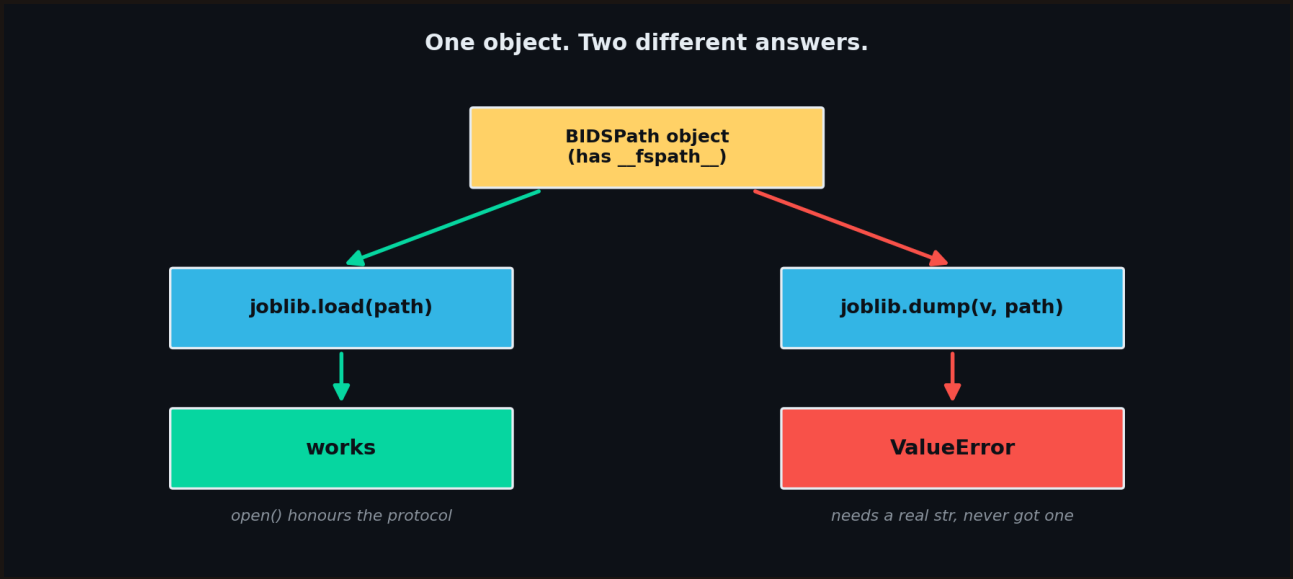
The bug, in one sentence

joblib.load() accepted any path-like object. **joblib.dump()** only accepted **str** and **pathlib.Path**.

So this happened:

```
from mne_bids import BIDSPath
import joblib

path = BIDSPath(root="./data/", subject="group", extension="pkl.gz")

joblib.load(path)          # works fine
joblib.dump(["TEST"], path)  # ValueError
```

Save fails. Load succeeds. Same object.



The same path object takes two routes through joblib and gets two different answers.

> The error message made it worse: "Second argument should be a filename or a file-like object" — followed by a perfectly valid filename. The user is told their filename is not a filename.

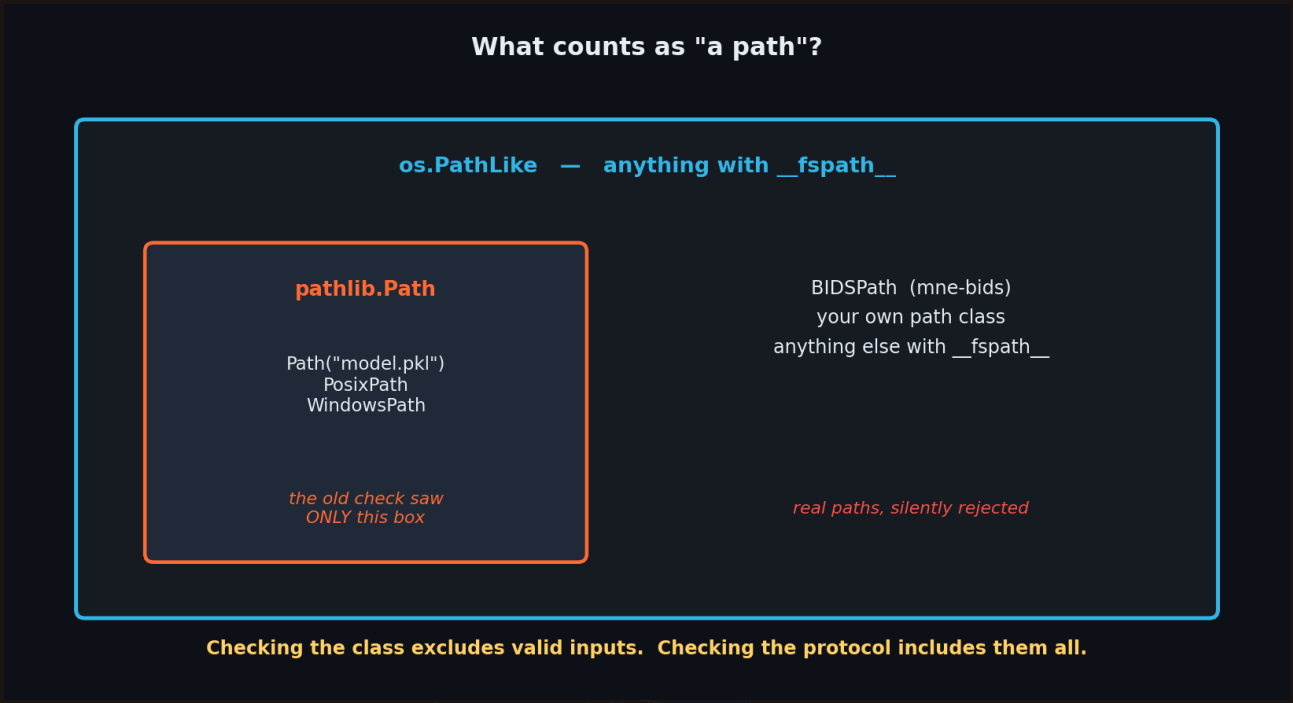
Who found it

Not me. A researcher called **skjerns** hit it in March 2026 while using **BIDSPath** from **mne-bids**, a library for neuroimaging data. He filed issue **#1784** with a clean reproducer.

It sat open for four months.

Why it happened

Python has a protocol for "this object is a path." Any class with a **__fspath__** method is path-like — **pathlib.Path** is just one implementation. **open()** accepts all of them.



pathlib.Path is one member of a much larger family. The old check only recognised the inner box.

joblib's **dump()** did not check the protocol. It checked one concrete class:

```
# the old code
if Path is not None and isinstance(filename, Path):
    filename = str(filename)
```

Look at that **Path is not None** guard. That is a fossil — from the days when **pathlib** might not exist in your Python. It had been true for a decade and nobody had reason to look at the line again.

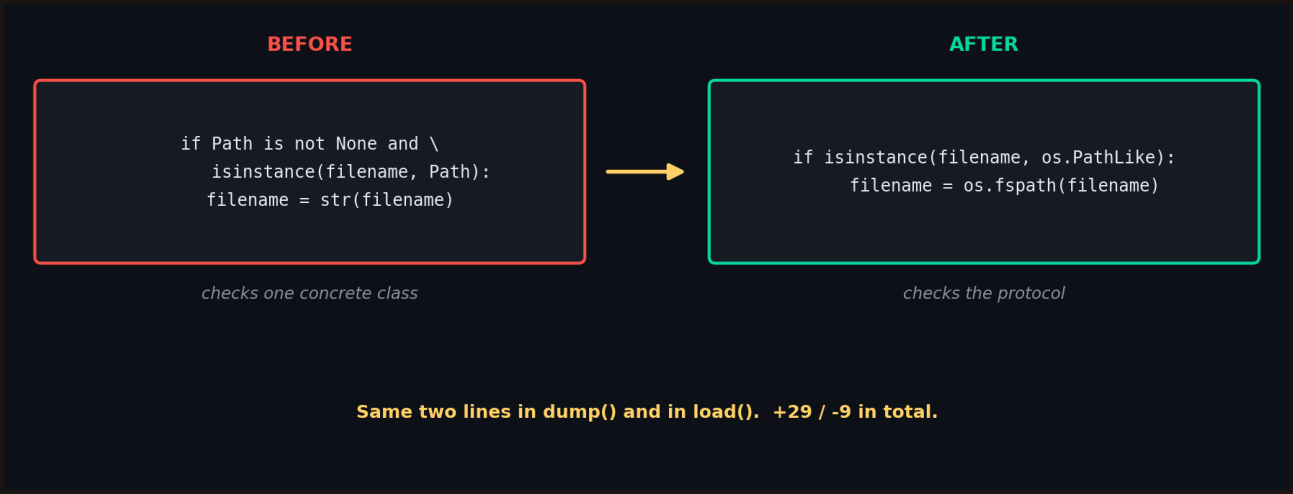
BIDSPath is path-like. It is not a **pathlib.Path**. So it failed this check, fell through to the next one, and raised.

> Then why did load() work? Because after this same check, load() hands the object to open() — and open() honours the protocol. dump() takes a different route and needs a real str. Same guard, two outcomes.

The fix

```
# the new code
if isinstance(filename, os.PathLike):
    filename = os.fspath(filename)
```

os.PathLike is the protocol. **os.fspath()** is the standard way to resolve it. The same two lines appear in **dump()** and in **load()**, so both were replaced — plus the now-unused **from pathlib import Path** import, and the docstrings that promised **pathlib.Path**.



The entire behaviour change: stop asking what class it is, start asking what it can do.

> The whole diff was +29/-9, and most of that was the test and the changelog. The behaviour fix is one word: Path becomes os.PathLike.

What made it merge

Four things, and none of them were clever code.

- The issue had **zero comments and no assignee** — nobody else was working on it, so there was no race
- It was a **real user's real problem**, not a style opinion someone could disagree with
- The fix used the **standard protocol**, so a maintainer did not have to weigh alternatives
- It shipped with a **test that fails without the fix** — a small class implementing **__fspath__**, no new dependency

The test

This is the part people skip, and it is the part that makes review easy:

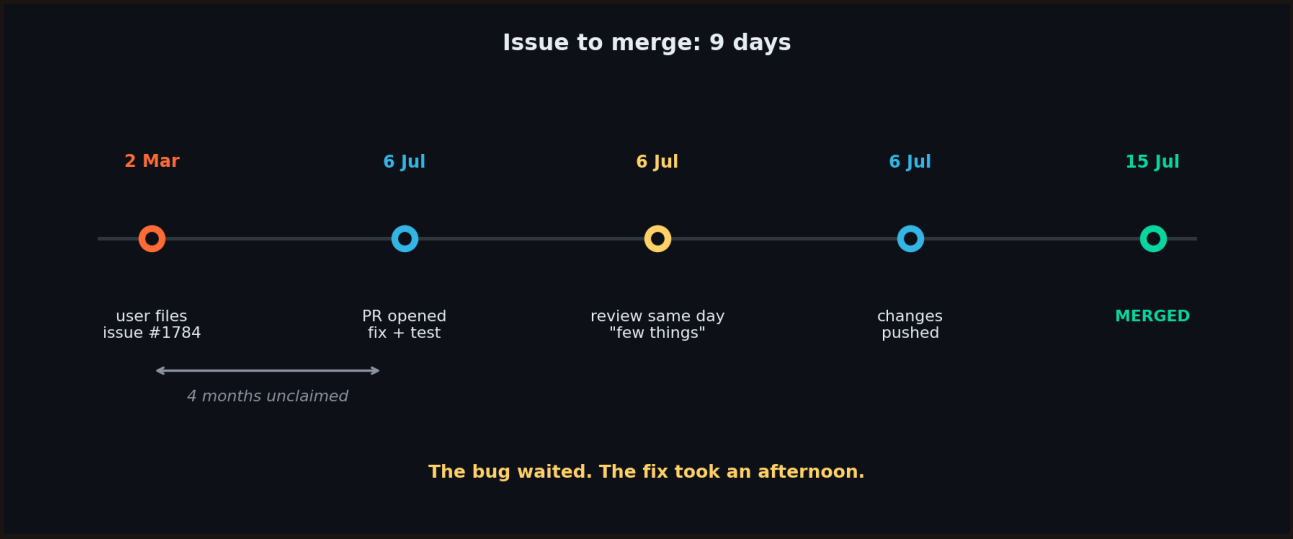
```
def test_dump_and_load_accept_os_pathlike(tmpdir):
    class CustomPath(os.PathLike):
        def __init__(self, path):
            self._path = path

        def __fspath__(self):
            return str(self._path)

    path = CustomPath(tmpdir.join("test_pathlike.pkl").strpath)
    value = {"key": [1, 2, 3]}
    numpy_pickle.dump(value, path)
    assert numpy_pickle.load(path) == value
```

No mock, no fixture file, no network, no new dependency. **CustomPath** is a stand-in for **BIDSPath** — the smallest object that satisfies the protocol. A reviewer reads it in ten seconds and sees exactly what broke.

> Notice what the test does not do: it does not import mne-bids. Never make a maintainer install a neuroimaging library to verify your fix. Reproduce the shape of the bug, not the user's whole stack.

The timeline

When	What
2 March	skjerns files issue #1784 with a reproducer
6 July	I hit the same inconsistency, check nobody has claimed it
6 July	Open PR #1812 — fix, test, CHANGES entry
6 July	Maintainer reviews the same day: "few things before merging"
15 July	Merged by Nanored4498



Four months of nobody claiming it, then nine days from PR to merge.

Nine days from open to merged, and the first review came back in hours. Not because I was fast — because the PR was small enough to review in one sitting.

The review, and what it teaches

The maintainer requested two changes. Neither was about the fix.

- A docstring line he wanted worded differently — he sent it as a **GitHub suggestion**, so accepting it was one click
- Some lines that overlapped with my *other* open PR, #1811. He asked: **"Could you reverse these changes that will be merged in your other PR?"**

That second one is the lesson. I had two PRs open on the same repo touching nearby lines, which forces the maintainer to think about merge order. Keep each PR to one concern and do not let them overlap.

> He closed the review with a smiley. Maintainers are not gatekeepers waiting to reject you — they are volunteers hoping your PR is easy to merge. Make it easy.

What I would tell you to copy

- **Contribute to a library you actually use.** I found this by reading joblib's source for an unrelated reason. That is a better filter than any "good first issue" label.
- **Check it is unclaimed before you write code.** Look at linked PRs and read the comments, not just the assignee field. Most wasted effort is a duplicate.
- **Reproduce it first.** If you cannot make it fail on your machine, you cannot prove you fixed it.
- **Write the test before the PR body.** If the test is hard to write, the fix is probably wrong.
- **Keep it small.** A one-line fix with a test merges. A refactor waits.

> The uncomfortable truth: my first thirty-five PRs were docstring additions. None of them merged. This one — a real bug, in a library I use, with a test — merged in nine days. Volume did not work. One real fix did.

What is next

Next video: the pypdf image bug. A PDF that decoded to the wrong colours, the maintainer telling me my first fix did not solve his case, and what I did about it.

> Try this: open the GitHub issues of one library you used this week. Filter to "bug", sort by oldest, and look for one with no linked PR. That is where your first merge is.